

Documentation

Dashboard for Visualizing SQL Data from the Archive



SOLARIS
NARODOWE CENTRUM
PROMIENIOWANIA
SYNCHROTRONOWEGO

Location: National Synchrotron Radiation Centre SOLARIS
Author: Karol Ożóg
Code: https://github.com/karolozog15/dashboard_solaris.git

A web application for browsing and visualizing archived measurement data stored in Microsoft SQL Server databases. It is written in Python using the Streamlit framework and the `pandas`, `pyodbc` and `plotly` libraries. The project also includes the `start.vbs` script, which makes it easier to launch the application on Windows.

Kraków, 2026

Contents

1	Preparing and Running the Application	1
1.1	Requirements	1
1.2	Project Files	1
1.3	Installation and Running from the Terminal	1
1.4	Installation Without Internet Access	1
1.5	Running via <code>start.vbs</code>	2
2	Description of the Application	2
2.1	Main Application Flow	2
2.2	Selecting the Database	2
2.3	Selecting Tables and Variables	2
2.4	Configuring Series	2
2.5	Time Range	3
2.6	Refreshing and Presenting Data	3
3	The <code>config.py</code> Module	3
3.1	Connection, Time and Limits	3
3.2	Series Colors	3
3.3	Icons	3
4	The <code>db.py</code> Module	3
4.1	Pooling and Creating Connections	4
4.2	Schema and Table Names	4
4.3	Retrieving Databases, Tables and Variables	4
4.4	Descriptive Variable Names	4
4.5	Fetching and Aggregating Data	4
4.5.1	Filtering and Computing Time	4
4.5.2	Determining Bounds and Buckets	5
4.5.3	Aggregating Values	5
4.6	Result Conversion in Pandas	5
4.7	Query Cache	5
5	The <code>state.py</code> Module	5
5.1	Initializing the Session State	5
5.2	Range Selection Modes	6
5.3	Remembering the Time Range	6
5.4	Adding, Removing and Resetting Series	6
5.5	Range Selection and Zoom History	6
5.6	Undoing Zoom	6
6	The <code>charting.py</code> Module	6
6.1	Chart Keys and Widget State	7
6.2	Time Conversion and Formatting	7
6.3	Y-Axis Ranges and Placement	7
6.4	Building the Plotly Figure	7
6.5	AVG, MIN and MAX Traces	7
6.6	Series Summary	7
6.7	Reading Chart Events	8
6.8	M1 and M2 Markers	8
6.9	Range Selection and Zoom	8
6.10	Mouse and Space Bar Control in <code>controls.html</code>	8
6.11	Streamlit Fragment and Refreshing	8

1 Preparing and Running the Application

The application is used for browsing and visualizing historical measurement data stored in Microsoft SQL Server. The interface is built with Streamlit, and the charts are created with Plotly.

1.1 Requirements

To run the application you need:

- Python 3.10 or newer,
- access to a Microsoft SQL Server instance with read permissions,
- the ODBC Driver 17 for SQL Server driver,
- Windows and Windows authentication for the connection configuration used in the project,
- the Python libraries: `streamlit`, `pandas`, `plotly` and `pyodbc`.

The `start.vbs` script requires Windows. The Streamlit application itself does not use VBScript, but the current database connection configuration uses Windows authentication.

1.2 Project Files

The main project files are:

- `app.py` – the interface and the main application flow,
- `db.py` – the SQL Server connection, queries and data aggregation,
- `charting.py` – charts, axes, markers and summaries,
- `state.py` – session state, series and time ranges,
- `config.py` – connection settings, limits and colors,
- `controls.html` – additional chart control using the mouse and the space bar,
- `start.vbs` – the launch script for Windows,
- `README.md` – short project documentation.

1.3 Installation and Running from the Terminal

In the project directory, create a virtual environment:

```
python -m venv .venv
```

Then install the dependencies:

```
.\.venv\Scripts\python.exe -m pip install streamlit pandas plotly pyodbc
```

Run the application with the command:

```
.\.venv\Scripts\python.exe -m streamlit run .\app.py
```

By default, Streamlit serves the application at:

```
http://localhost:8501
```

The SQL Server address and the default database name are defined in `config.py`. The details of creating the connection and the SQL queries can be found in `db.py`.

1.4 Installation Without Internet Access

On a computer with Internet access, you can download the packages and their dependencies into the `packages` directory:

```
py -m pip download -d .\packages streamlit pandas plotly pyodbc
```

The `packages` directory must be copied to the target computer. The packages can then be installed there without Internet access:

```
.\.venv\Scripts\python.exe -m pip install --no-index --find-links=.\packages streamlit pandas plotly pyodbc
```

The packages downloaded for offline installation must match the Python version and the architecture used on the target computer.

1.5 Running via start.vbs

On Windows, the application can be launched with the `start.vbs` file. The script expects the interpreter:

```
.venv\Scripts\python.exe
```

and the `app.py` file. It checks whether port 8501 is in use. If the port is free, it starts Streamlit and opens the local application address in the browser.

The script writes the local address and the detected network address to the `link.txt` file. The availability of the network address depends, among other things, on the computer configuration and the firewall.

If port 8501 is in use, the script checks the process name, the interpreter path and the command line. The process is terminated only if it is recognized as the expected instance of the application. Otherwise the script displays a warning and does not kill the process. After the old instance has been successfully terminated, a new one is started.

2 Description of the Application

2.1 Main Application Flow

The `app.py` file is the main entry point of the application. It is responsible for configuring the page, displaying the interface, retrieving user settings, fetching data and passing it to the charting module.

The main flow includes:

1. configuring the Streamlit page and initializing the session state,
2. retrieving the list of databases and selecting a database,
3. retrieving the tables available in the selected database,
4. retrieving descriptive variable names, if available,
5. selecting and configuring data series,
6. selecting the time range and the refresh frequency,
7. fetching and aggregating the data,
8. displaying the chart, the summary and an optional data table.

The Streamlit page is displayed in a wide layout. After the session state is initialized, the application shows the dashboard title and description.

2.2 Selecting the Database

The list of available databases is retrieved by the `load_databases()` function from the `db.py` module. By default, the database specified by `DEFAULT_DB_NAME` is selected, currently `Archives`. If it is not on the list, the application selects the first available database.

If the list of databases cannot be retrieved or is empty, the application displays an appropriate message and stops further execution.

After the database is changed, the `reset_series()` function is called. It removes the series configurations, because the tables and variables selected earlier may not exist in the new database.

2.3 Selecting Tables and Variables

After a database is selected, the application retrieves the list of tables. They are presented in the `SCHEMA.TABLE` format, for example `dbo.POMIARY`. After a table is selected, the available values of the `VARIABLE` column are retrieved.

Descriptive variable names are optionally retrieved from the table specified by `NAZWY_TABLE` in `config.py`. If retrieving the mapping fails, the application still works and shows only the variable identifiers.

2.4 Configuring Series

The user can select up to four series. For each series the user configures:

- the table and the variable,
- an automatic or manual Y-axis range,
- the lower and upper Y-axis limits in manual mode.

Series can be added and removed in the side panel. Each series has a separate UUID identifier, so the widget settings of one series do not conflict with those of the others.

2.5 Time Range

The application offers three modes of specifying the range:

- **Ostatnie dyskretne** (Last discrete) – selecting a predefined interval,
- **Ostatnie ciągłe** (Last continuous) – entering the number of seconds, minutes, hours and days back,
- **Własny zakres** (Custom range) – selecting the date and time of the start and the end.

The predefined ranges are 10 minutes, 1 hour, 1 day, 1 week, 1 month (30 days) and 1 year (365 days). The default predefined range is 1 hour.

If the start of the range is later than the end, the application displays an error and does not fetch data. Selecting a fragment of the chart can change the current range; the undo button restores the previous zoom level or the initial range.

2.6 Refreshing and Presenting Data

Data can be refreshed manually or periodically: every 1, 5, 15 or 30 minutes. The “Odśwież teraz” (Refresh now) button clears the measurement data cache and reruns the application.

The chart shows AVG values and, optionally, also MIN and MAX. Each series has its own Y axis. Below the chart there is a table summarizing the series. A table of aggregated data can be enabled separately; it contains at most the 1000 most recent records per series.

3 The config.py Module

The `config.py` file contains the most important configuration constants: the database connection settings, the default database, application limits, the cache retention time and the series colors.

3.1 Connection, Time and Limits

The database configuration includes:

```
DB_SERVER = r"ZENON14WIN\ZENON_2022"
DEFAULT_DB_NAME = "Archives"
DB_DRIVER = "ODBC Driver 17 for SQL Server"
NAZWY_TABLE = "dbo.WODA_VARIABLES"
```

The time zone is defined as a fixed UTC+1 offset. It is not a time zone that automatically accounts for the switch between daylight saving and standard time.

The most important limits and cache times are:

```
MAX_PLOT_POINTS = 1_000
MAX_TABLE_ROWS = 1_000
MAX_SERIES = 4
```

```
DATA_CACHE_TTL = 300
VARIABLE_CACHE_TTL = 600
TABLE_CACHE_TTL = 600
```

Measurement data is kept in the cache for 300 seconds, and the lists of databases, tables and variables for 600 seconds. The detailed cache entry limits are set directly in the function decorators in `db.py`.

The `AXIS_GAP` and `MAX_DOMAIN_START` constants affect the placement of the additional Y axes and the width of the chart area.

3.2 Series Colors

The `VARIABLE_COLORS` list contains the colors used to distinguish the series. The `get_series_color(index)` function selects a color based on the series index. If the index exceeds the number of colors, the palette is used cyclically.

3.3 Icons

The paths to the icons used by Streamlit, so that they can be used in other places, are stored in `config.py`.

4 The db.py Module

The `db.py` module is responsible for the connection to SQL Server, retrieving database metadata, querying measurement data and aggregating it. The results of the data-fetching functions are stored in the Streamlit cache.

4.1 Pooling and Creating Connections

The module enables connection pooling in the `pyodbc` library:

```
pyodbc.pooling = True
```

The connection is created by the `get_connection(database)` function. It uses the server and driver settings from `config.py`. If no database name is given, the `master` database is used.

The connection uses Windows authentication (`Trusted_Connection=yes`), does not force encryption (`Encrypt=no`) and has a 10-second connection timeout. Functions that execute queries close the connection in a `finally` block.

4.2 Schema and Table Names

The `quote_identifier(value)` function prepares a schema or table name for insertion into an SQL query. It strips leading and trailing spaces, rejects an empty name and names containing the `]` character. A valid name is enclosed in square brackets; for example, `dbo` becomes `[dbo]`.

SQL object names are not passed as query parameters, so they are validated before being inserted into the query. Values such as the variable identifier and the time limits are passed separately as parameters.

4.3 Retrieving Databases, Tables and Variables

The `load_databases()` function retrieves databases with the `ONLINE` state from the `sys.databases` system table. The condition `database_id > 4` skips the standard system databases `master`, `tempdb`, `model` and `msdb`. The function returns a list of database names.

The `load_tables(database)` function retrieves base tables from `INFORMATION_SCHEMA.TABLES`, skipping views. It returns a `DataFrame` with the columns `TABLE_SCHEMA` and `TABLE_NAME`.

The `load_variables(database, table_schema, table_name)` function retrieves the unique, non-empty values of the `VARIABLE` column from the selected table and returns them in alphabetical order.

4.4 Descriptive Variable Names

The `load_variable_names(database)` function retrieves the `VARIABLE` and `NAME` columns from the table specified by `NAZWY_TABLE`. The result is converted into a dictionary in which the variable identifier is the key and its descriptive name is the value.

The `get_variable_label(variable, names_map)` function returns a label in the form `Name (IDENTIFIER)` if a descriptive name is available. Otherwise it returns the identifier alone.

4.5 Fetching and Aggregating Data

The main function that fetches measurements is `load_data()`. It takes the database name, the schema and table, the variable identifier, and the start and end of the range as Unix timestamps in seconds.

The SQL query consists of four stages written as CTEs:

1. `filtered` – filtering records and computing the time in milliseconds,
2. `bounds` – determining the time bounds of the fetched set,
3. `bucketed` – assigning records to intervals,
4. `aggregated` – computing statistical values in each interval.

4.5.1 Filtering and Computing Time

The `filtered` CTE selects records of the given variable from the range:

```
VARIABLE = ?
TIMESTAMP_S >= ?
TIMESTAMP_S <= ?
```

The values are passed as parameters to `pandas.read_sql`. This removes the need to manually insert user values into the query.

The query also fetches the columns `CALCULATION`, `STATUS`, `GUID` and `STRVALUE`. The time in seconds and milliseconds is combined into a helper value:

```
TIMESTAMP_S * 1000 + COALESCE(TIMESTAMP_MS, 0)
```

If the `TIMESTAMP_MS` value is empty, zero is assumed. Range filtering in the query is done on the `TIMESTAMP_S` column, that is, with one-second accuracy; the milliseconds are used later, among other things, for sorting and for determining the time of a point.

4.5.2 Determining Bounds and Buckets

The **bounds** CTE determines the earliest and latest time in the filtered set. Each record is then given a bucket number computed proportionally to its position within the range:

$$bucket = \left\lfloor \frac{t - t_{\min}}{t_{\max} - t_{\min}} \cdot N \right\rfloor$$

where t is the record time in milliseconds, $t_{\min}$ and $t_{\max}$ denote the time bounds, and N is the value of `MAX_PLOT_POINTS`, currently equal to 1000.

Bucket numbers start at zero. According to the current formula, the latest record can receive the number 1000, so the number of non-empty buckets can reach 1001. If all records have the same time, they fall into bucket 0. A condition in the query protects the computation against division by zero.

4.5.3 Aggregating Values

The **aggregated** CTE groups records by variable and bucket number. Rows with an empty **VALUE** are skipped. For each bucket the following are computed:

```
AVG(VALUE) AS VALUE_AVG
MIN(VALUE) AS VALUE_MIN
MAX(VALUE) AS VALUE_MAX
```

The columns **CALCULATION**, **STATUS**, **GUID** and **STRVALUE** are represented by the **MIN** value in the group. For these columns it is not a measurement statistic, but a way of choosing a single value to represent the bucket.

The query also stores the earliest and latest time in each bucket. The time of the resulting point is determined as the midpoint between these values and then split again into seconds and milliseconds. The result is sorted chronologically by both columns.

4.6 Result Conversion in Pandas

If the query returns no records, the function returns an empty **DataFrame** with the expected columns, including **time**, **VALUE_AVG**, **VALUE_MIN** and **VALUE_MAX**.

For a non-empty result the function:

- converts the time and value columns to numeric types,
- creates the **time** column from seconds and milliseconds,
- converts the time according to **LOCAL_TZ**,
- removes rows without a valid time or AVG value,
- sorts the data chronologically and resets the index.

The resulting **time** column has no time zone information. In the current configuration, **LOCAL_TZ** means a fixed UTC+1 offset.

4.7 Query Cache

The functions that fetch databases, tables, variables, variable names and data use **st.cache_data**. Entries depend on the function arguments, e.g. the database name, table, variable and time range.

The cache of the lists of databases, tables and variables is valid for 600 seconds, and the cache of measurement data for 300 seconds. The manual refresh button in **app.py** clears the cache of the **load_data()** function.

5 The state.py Module

The **state.py** module manages the Streamlit session state, the series configuration and the time range. It also handles range selection on the chart and undoing successive zoom levels.

5.1 Initializing the Session State

Streamlit re-executes the script after many user actions. Values that should survive such a rerun are stored in **st.session_state**.

The **init_session_state()** function sets a default value only if the given key does not yet exist. On first launch it creates one series with a UUID identifier and sets the initial range to the last hour.

The default mode is **Ostatnie dyskretne** (Last discrete), and the default predefined range is **1 godzina** (1 hour). The session state also stores, among other things, the series configurations, widget values, the current range, the zoom state, the zoom history and the chart key version.

5.2 Range Selection Modes

The application supports three modes:

- **Ostatnie dyskretne** (Last discrete) – selecting a predefined interval,
- **Ostatnie ciągłe** (Last continuous) – entering the number of seconds, minutes, hours and days back,
- **Własny zakres** (Custom range) – entering the date and time of the start and the end.

The predefined ranges stored in `QUICK_RANGES` are 10 minutes, 1 hour, 1 day, 1 week, 1 month (30 days) and 1 year (365 days).

The current range and the range remembered before zooming are stored separately:

```
time_start, time_end
time_start_original, time_end_original
```

5.3 Remembering the Time Range

The `insert_time(start_time, end_time)` function updates the widget values according to the selected mode. In discrete mode it selects the predefined range closest to the given length. In continuous mode it breaks the range length into days, hours, minutes and seconds. In custom mode it updates the date and time fields.

Regardless of the mode, the function saves the range as both the current and the remembered one. It is used, among other places, when adding and removing series, in order to preserve the set time after a Streamlit rerun.

5.4 Adding, Removing and Resetting Series

The `add_series(max_series)` function adds a new series if the number of series has not reached the limit. Each new series receives a UUID identifier.

The `remove_series(series_id)` function removes the selected series, its configuration and the stored table and variable widget keys. Before a series is added or removed, if zoom is not active, the current time range is preserved.

The `reset_series()` function clears the configurations of all series and creates one new series. It is used after the database is changed. It does not reset the time range by itself.

5.5 Range Selection and Zoom History

The `handle_zoom_box_select(box_x)` function handles the rectangular selection passed from the chart. It expects at least two time values. If the selection is identical to the last processed one, the function skips it so as not to handle the same event again during the next rerun.

The ends of the selection are ordered chronologically. On the first selection, the previous range mode is remembered and the zoom history is cleared. On a subsequent selection, the current range is added to `zoom_history`, after which the new range is set.

The function marks zoom as active and increments `chart_key_version`. It returns `True` if the range was changed, and `False` if the selection is invalid or has already been processed.

A selection changes the range used to fetch data; it is not merely a visual magnification of the existing points.

5.6 Undoing Zoom

The `undo_zoom()` function undoes one zoom level. If `zoom_history` contains an earlier range, it is popped from the stack and restored.

When the history is empty, the function restores the time selection mode and the range from before the first zoom. It then disables zoom, clears the information about the last selection and increments `chart_key_version` in order to refresh the state of the chart component.

Validation of whether the start of the selected range is not later than its end is done in `app.py`.

6 The charting.py Module

The `charting.py` module is responsible for building the Plotly chart and rendering the chart-related interface. It supports multiple series and Y axes, AVG and MIN/MAX traces, M1 and M2 markers, the data summary, chart events and periodic refreshing.

The module works together with `state.py`, which handles the range change after a selection, and with `controls.html`, which adds additional control using the mouse and the space bar.

6.1 Chart Keys and Widget State

The `_widget_key()` and `_chart_keys()` functions create widget keys and state values for the MIN/MAX options, axis placement, markers and the chart component. The default chart identifier is `main_chart`; with multiple charts, each should have its own `chart_key`.

The component key takes `chart_key_version` into account. The same value is set as the `uirevision` of the Plotly figure. Changing the version allows the chart state to be refreshed after the range changes or zoom is undone. On an ordinary rerun, the Plotly interaction state may be preserved.

6.2 Time Conversion and Formatting

The `_to_naive_timestamp()` function converts a value to a `pandas.Timestamp` and removes the time zone information. An invalid or empty time is converted to `None`.

The `_to_python_datetime()` function returns a standard `datetime.datetime` object, used, among other things, for handling selections. The `_format_dt()` function formats the time as `YYYY-MM-DD HH:MM:SS.mmm`.

6.3 Y-Axis Ranges and Placement

The `_get_y_range()` function reads the Y range for a given series. If automatic scaling is active or the manual range is invalid, it returns `None`. Otherwise it returns a pair of limit values, provided that the minimum is smaller than the maximum.

The `_axis_for_series()` function assigns series to axes:

- the first series uses the axis on the left side,
- the second series by default uses the axis on the right side,
- subsequent series receive additional axes on the left side,
- the “Wszystkie osie po lewej” (All axes on the left) option places all axes on the left side.

The additional axes overlay the main axis and are positioned using the `AXIS_GAP` and `MAX_DOMAIN_START` parameters. Axis titles and labels take the color assigned to the series.

6.4 Building the Plotly Figure

The `_build_figure()` function creates the figure and adds the data traces to it. For each series the following columns are required:

```
time
VALUE_AVG
VALUE_MIN
VALUE_MAX
```

Empty series or series without the required columns are skipped. For a valid series a Y axis is created, along with the main AVG trace and — if the user has enabled the corresponding option — the MIN and MAX traces.

The chart is 700 pixels high, uses the dark Plotly template, and its X axis represents time. The X-axis range comes from the `start_time` and `end_time` arguments. The legend is laid out horizontally below the chart.

The chart uses, among other things, the closest-point hover mode, click and selection handling, and the initial zoom drag mode.

6.5 AVG, MIN and MAX Traces

The `_add_series_trace()` function adds a trace as a `go.Scattergl`. It configures its data, color, line width and style, the assigned Y axis, the legend, the `customdata` helper data and the tooltip format.

The AVG trace is shown as a line with markers and appears in the legend. After the “Pokaż zakres MIN/MAX” (Show MIN/MAX range) option is enabled, semi-transparent dotted MIN and MAX lines are added. These traces are hidden in the legend. The MIN/MAX option is enabled by default for at most two series.

In the current code, the MAX trace is drawn from the `VALUE_MAX` column, but passes the label MIN to the tooltip. The tooltip of this trace may therefore incorrectly describe the value as MIN.

6.6 Series Summary

Below the chart a summary table is displayed containing the series and table label, the last AVG value, the mean of the AVG values, and the minimum and maximum determined from the aggregated MIN and MAX columns.

The summary is computed from the data passed to the chart, that is, data already aggregated by the SQL query.

6.7 Reading Chart Events

The `_get_attr()` function reads an attribute from both a dictionary and an object returned by Streamlit. The functions `_get_selection()`, `_get_selection_points()` and `_get_box_selections()` retrieve selected points and rectangular selections. If data is missing, empty lists or default values are returned.

Each point receives the index of its series in `customdata`. The `_extract_series_idx()` function reads this index. If it is missing, the `_series_index_from_curve()` function tries to determine the series from the trace number. In this mode, one line per series is assumed without MIN/MAX, or three traces per series with MIN/MAX.

6.8 M1 and M2 Markers

Marker mode is enabled by the “Markery klikane” (Clickable markers) option. The first clicked point sets M1, the second sets M2. Each subsequent click moves the previous M2 to the M1 position and sets the new point as M2.

The time, value, trace number and series index are stored. The signature of the last click prevents the same event from being processed multiple times during Streamlit reruns.

M1 is displayed in green and M2 in pink. For each marker a label is added, along with a vertical line indicating the time and a horizontal line indicating the value. Markers are assigned to the Y axis of the appropriate series.

The marker table shows the time and value of the selected points. If M1 and M2 belong to the same series, the following are also given:

$$\Delta Y = Y_{M2} - Y_{M1}$$

$$\Delta t = t_{M2} - t_{M1}$$

$$\frac{\Delta Y}{\Delta t}$$

The time Δt is expressed in seconds. The quotient is not computed if both markers have the same time. When the markers belong to different series, their values are presented separately, without computing the difference between them. The “Wyczyść markery” (Clear markers) button removes M1 and M2. The markers are also cleared when marker mode is disabled.

6.9 Range Selection and Zoom

The chart is rendered by `st.plotly_chart()` with support for point and area selections. The rectangular selection is passed to the `handle_zoom_box_select()` function from the `state.py` module. If the range has been accepted, the application performs a full rerun and fetches the data again for the new interval.

Ordinary Plotly zoom changes the chart view on the browser side. By itself it does not change the range for which the application fetches data. Data is fetched again after a Box Select selection is handled.

6.10 Mouse and Space Bar Control in `controls.html`

The `install_controls()` function loads the `controls.html` file and embeds its contents using `components.html()`. The file is opened from a path relative to the current working directory, so it should be available when the application is launched.

The JavaScript script adds event handling to the parent document of the page. A flag in this document prevents the handlers from being installed multiple times. Above the chart, the browser’s standard context menu is also blocked.

The mouse controls are as follows:

- the left button performs ordinary Plotly zoom,
- the right button switches the chart to Box Select,
- the middle button switches the chart to Pan, that is, moving the view.

For the right and middle buttons the script switches the Plotly mode and then passes on the mouse events required for selection or panning. After the button is released, the chart mode returns to zoom.

Pressing the space bar activates the automatic view-fitting button in the Plotly toolbar. The mouse and space bar handling is located in the HTML file; passing the selected range on to data fetching is done in Python.

6.11 Streamlit Fragment and Refreshing

The `make_chart_fragment()` function wraps chart rendering in a Streamlit fragment. The refresh interval is passed in seconds; the value `None` disables periodic refreshing.

The fragment refreshes the part of the interface containing the chart, independently of the rest of the page. Manual refresh is handled in `app.py`: it clears the cache of the `load_data()` function and reruns the application in order to fetch data from SQL Server.